

More Raster, More Vector, More Raster-Vector Interactions

Alexander Abuabara

DAEN 489 ISEN 489 Spring 2026

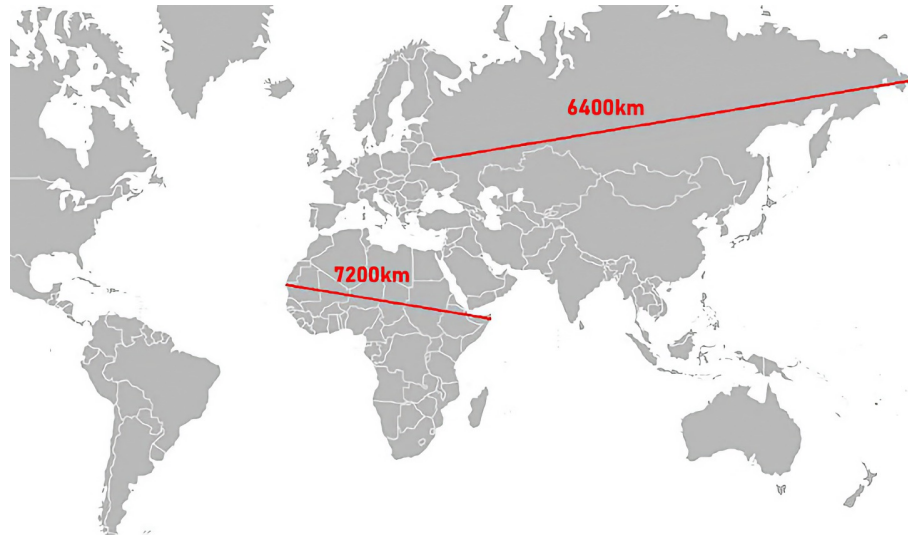


ENGINEERING
TEXAS A&M UNIVERSITY

Outline

- Three questions that analyze the integration, conversion, and spatial relationships between discrete vector features and continuous raster surfaces.
- You don't have specific instructions on how to answer each question, but you should use knowledge from the classes.
 - One of your goals is to create the data analysis workflow and be able to explain it with a graph, diagram, or pseudo-code.
- **Q1:** Verify a possible internet hoax
- **Q2:** Estimate snacks needed
- **Q3:** Maybe a more accurate estimate of snacks

The other day I saw the map below on the internet (reddit).

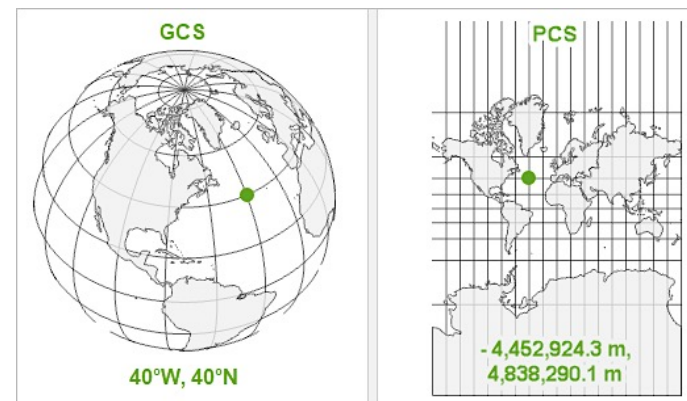


- Q1: Using the tools you have learned so far,
can you roughly verify if it is a hoax or real?
 - Provide a brief commentary and your conclusion (hoax or real).

Recap

- Geographic vs Projected Coordinate Systems:
 - A GCS is round, and so records locations in angular units (usually degrees).
 - A PCS is flat, so it records locations in linear units (usually meters).

Coordinate System Details		
Projected Coordinate System	Fuller (world)	Tell me how to draw the earth on a flat surface!
Projection	Fuller	
WKID	54050	
Authority	Esri	
Linear Unit	Meters (1.0)	
False Easting	0.0	
False Northing	0.0	
Option	0.0	
Geographic Coordinate System	WGS 1984	Tell me where on the earth the data should draw!
WKID	4326	
Authority	EPSG	
Angular Unit	Degree (0.0174532925199433)	
Prime Meridian	Greenwich (0.0)	
Datum	D WGS 1984	
Spheroid	WGS 1984	
Semimajor Axis	6378137.0	
Semiminor Axis	6356752.314245179	
Inverse Flattening	298.257223563	



Notes:

- WKID (Well-Known ID) is a unique identifier number for the PCS, defined by its authority.
- For example, [EPSG](#) (European Petroleum Survey Group).
- Coordinates in a PCS are recorded in a linear unit (often meters :)

GPS Coordinate

- Identifies a location, 2 values:
 - **Longitude:** East–West location
 - **Latitude:** North–South location
- Coordinate order in most GIS systems: (longitude, latitude)
- Common beginner mistake: reversing coordinates.
 - Example: **Texas A&M University**
-96.3398, 30.6212

```
library(sf)

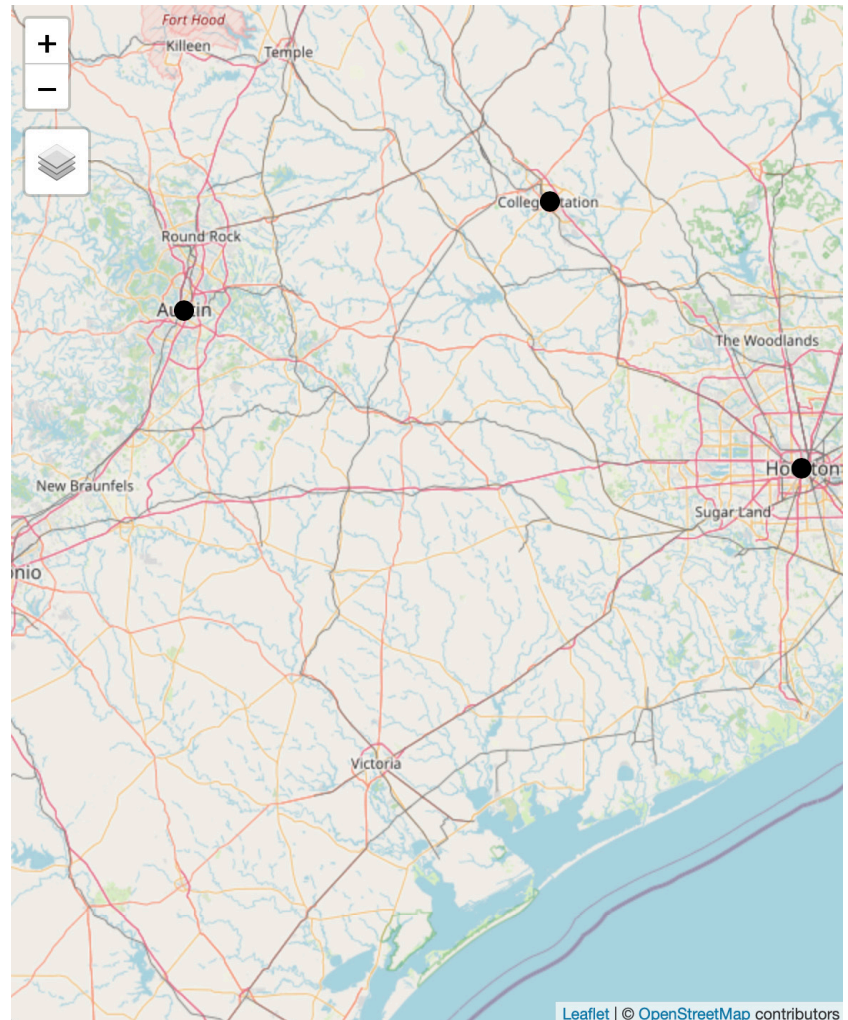
lon <- -96.3398
lat <- 30.6212

# geometry point
point_geom <- st_point(c(lon, lat))

# convert Geometry to an sf object
point_sf <- st_sf(
  location = "Texas A&M",
  geometry = st_sfc(point_geom),
  crs = 4326 # standard GPS coordinate system
)
```

Multiple GPS Points

```
locations <- data.frame(  
  name = c("Texas A&M", "Austin", "Houston"),  
  lon = c(-96.3398, -97.7431, -95.3698),  
  lat = c(30.6212, 30.2672, 29.7604)  
)  
  
points_sf <- st_as_sf(  
  locations,  
  coords = c("lon", "lat"),  
  crs = 4326  
)  
  
plot(st_geometry(points_sf))  
  
library(tmap)  
tmap_mode("view")  
  
tm_shape(points_sf) +  
  tm_dots(col = "maroon", size = 1)
```



ggplot (based on [The Grammar of Graphics](#), but applied to spatial data)

- **ggplot2** is like a stack of transparent pancakes. Each line of code adds a new layer:
 - The Base
 - The Map
(pulls OpenStreetMap tiles)
 - The Points
 - The Labels
 - The Style

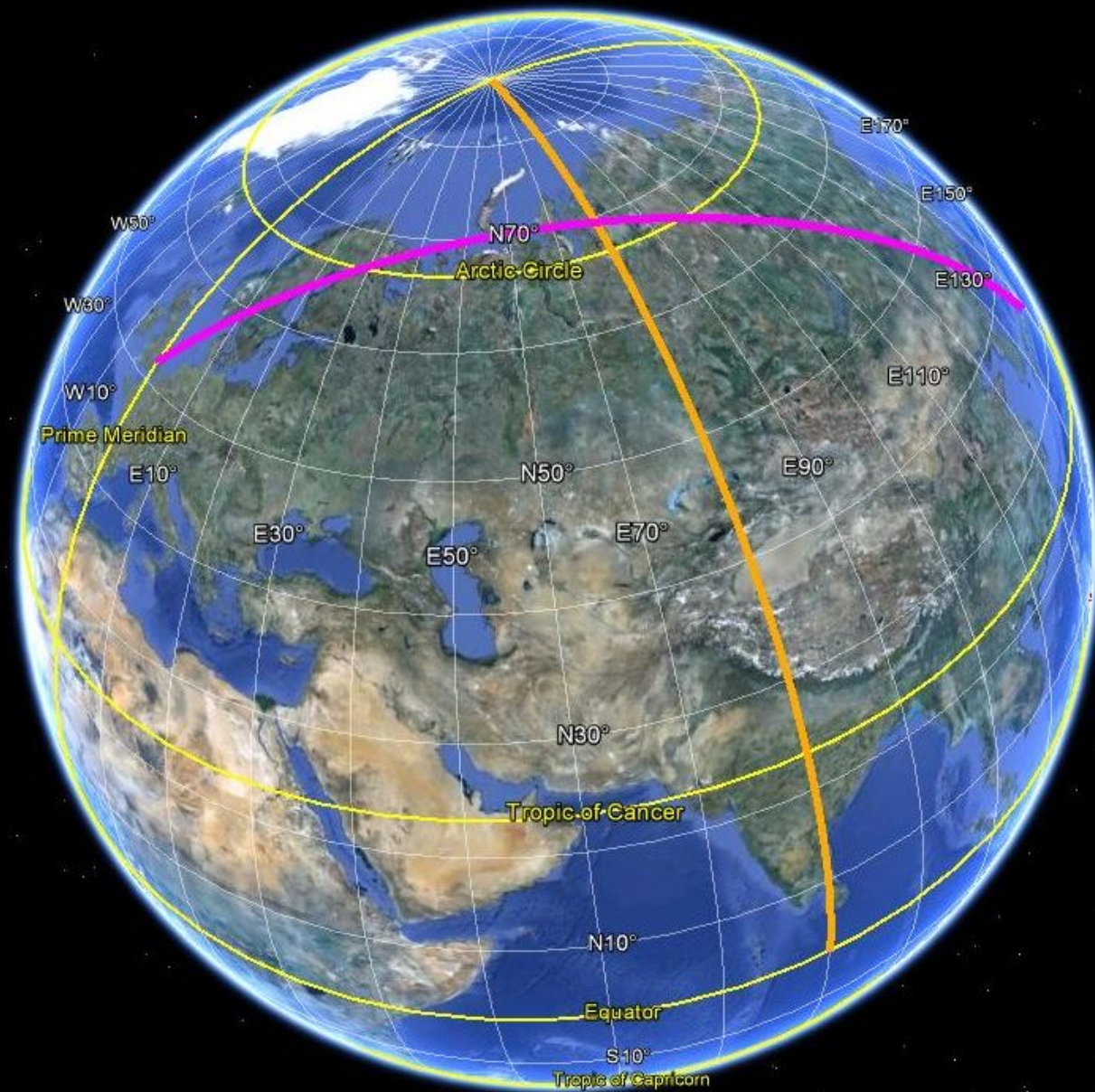
If the map looks "squished," R might be confused about the aspect ratio

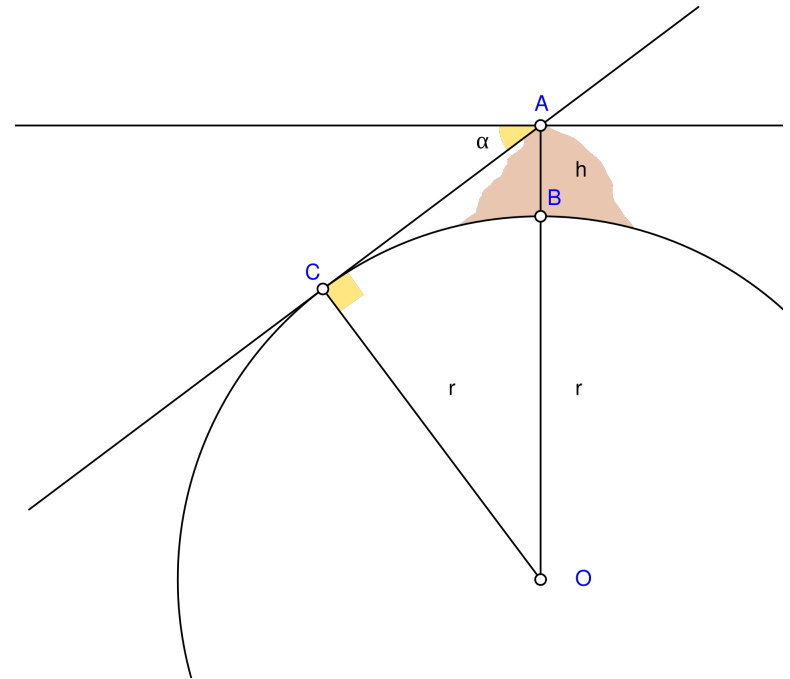
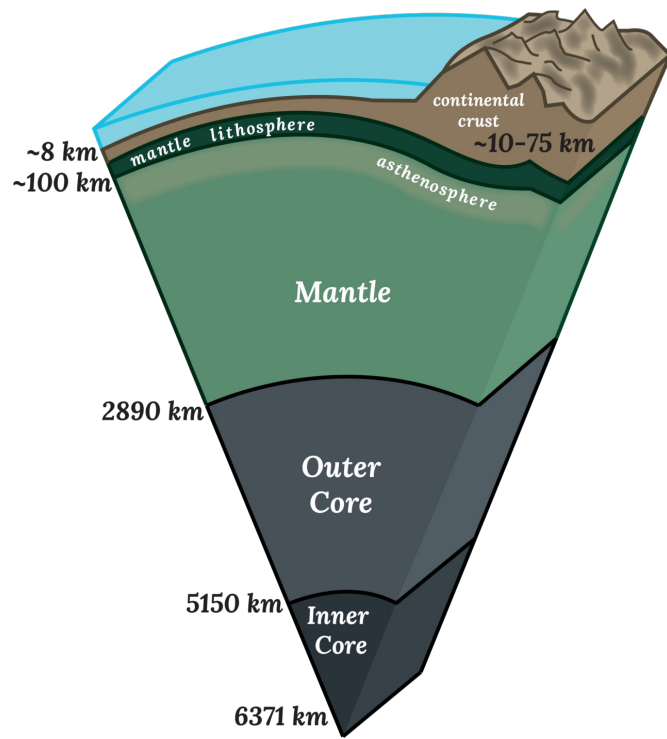
Adding **+ coord_sf()** at the end ensures that 1 degree North looks the same as 1 degree East

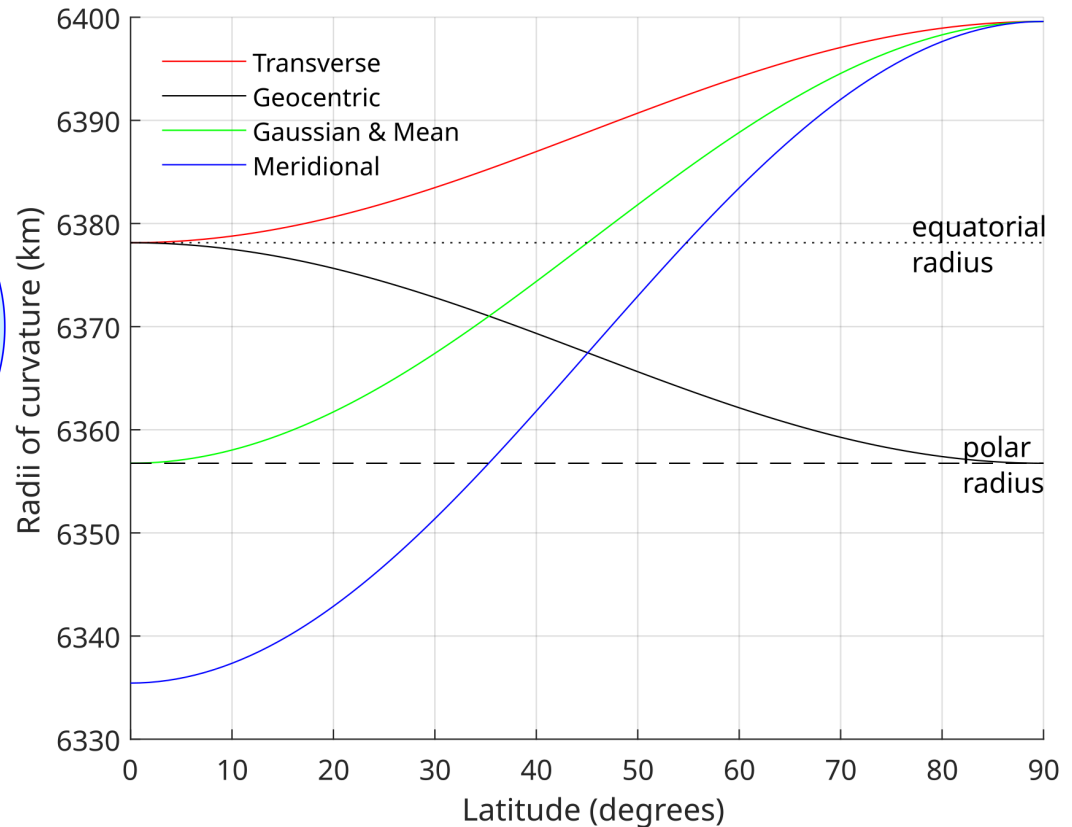
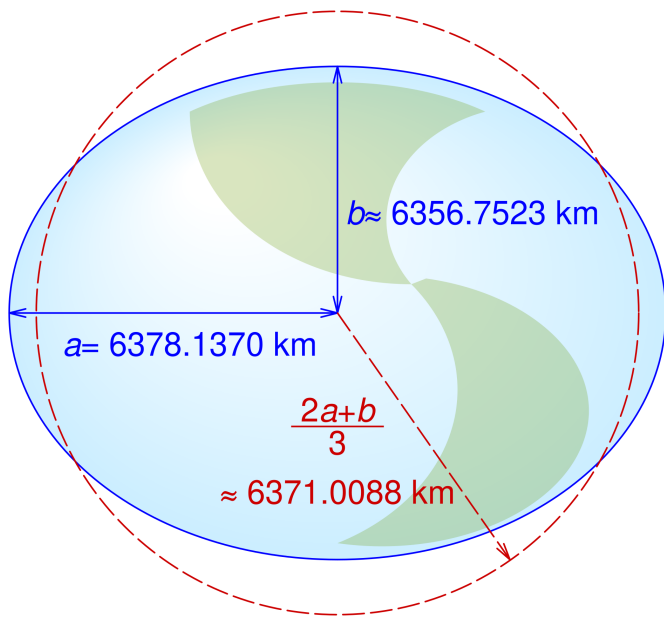
```
library(ggplot2)

library(ggspatial)

ggplot(data = points_sf) +
  annotation_map_tile(type = "osm", zoom = 15) +
  geom_sf(color = "#500000", size = 4) +
  geom_text(data = locations,
            aes(x = lon, y = lat, label = name),
            nudge_y = 0.0005,
            fontface = "bold") +
  theme_minimal()
```







- Earth radius is the distance from the center of Earth to a point on or near its surface.
- Approximating the figure of Earth by an **Earth spheroid** (an oblate **ellipsoid**), the radius ranges from a maximum (equatorial radius, denoted a) of about 6,378 km (3,963 mi) to a minimum (polar radius, denoted b) of nearly 6,357 km (3,950 mi).
- A globally-average value is usually considered to be 6,371 kilometers (3,959 mi) with a 0.3% variability ($\pm 10 \text{ km}$).

Distance Between Two Points

- The `st_distance()` function calculates the shortest distance between two geometries.
- If your data is in a geographic coordinate system (like WGS84, EPSG:4326), it automatically calculates the **great-circle distance** in meters. *(more on slide 14)*

```
st_distance(  
  points_sf[1,], points_sf[2,]  
)  
  
library(units)  
  
set_units(  
  st_distance(  
    points_sf[1,], points_sf[2,]),  
  "miles")
```

Working with Data Frames

- If you have a set of points in an sf object, **st_distance** will return a distance matrix comparing every point in the first set to every point in the second.

```
# Calculate distances between all  
# points in 'cities_sf'
```

```
dist_matrix <-  
  st_distance(cities_sf)
```

```
# To get the distance to the  
# nearest neighbor only:
```

```
min_dist <-  
  apply(st_distance(cities_sf),  
        1, function(x) min(x[x > 0]))
```

?apply

Pairwise Distance (Row-by-Row)

- If you have two columns of points and want to calculate the distance between Point A1 and Point B1, A2 and B2, etc. (rather than a full matrix), use **by_element = TRUE**

```
distances <-  
  st_distance(  
    set_p1, set_p2,  
    by_element = TRUE)
```

```
? st_distance
```

Important

- The output is usually a **units** object.
 - You can convert it to a standard numeric value or change units using the **units** library:
 - **as.numeric(dist)** removes the "m" label.
 - **units::set_units(dist, km)** converts meters to kilometers.

```
# Calculate distances between all
# points in 'cities_sf'

dist_matrix <-
  st_distance(cities_sf)

# To get the distance to the
# nearest neighbor only:

min_dist <-
  apply(st_distance(cities_sf), 1,
    function(x) min(x[x > 0]))
```

Great Circle vs. Euclidean

- If your CRS is geographic (degrees), **sf** uses the [S2 geometry library](#) to calculate distance on a sphere/ellipsoid.

By the way, the name "S2" is derived from the mathematical notation for the unit sphere, S^2 .

S2 approaches this problem by working exclusively with spherical projections. As the name implies, spherical projections map points on the Earth's surface to a perfect mathematical sphere

Such mappings still create some distortion, of course, because the Earth is not quite spherical—but as it turns out, the Earth is much closer to being a sphere than a plane.

With spherical projections, it is possible to approximate the entire Earth's surface with a maximum distortion of **0.56%**.

Spherical projections preserve the correct topology of the Earth; there are no singularities or discontinuities.

Great Circle vs. Euclidean

- If your CRS is geographic (degrees), `sf` uses the S2 geometry library to calculate distance on a sphere/ellipsoid.
- If your CRS is projected (meters/feet), it uses standard Euclidean geometry (Pythagorean theorem).

When your data is in a geographic CRS (like EPSG:4326/WGS84), the coordinates are in degrees (Latitude and Longitude). Because the Earth is roughly a sphere (or an ellipsoid), the shortest distance between two points isn't a straight line through the Earth, but a curve along the surface. This is the Great Circle distance. Use to long distances (e.g., flight paths from New York to London).

When your data is in a projected CRS (like UTM or State Plane), the coordinates are in linear units like meters or feet. A projection "flattens" the Earth's curve onto a 2D plane. Once the map is flat, we can use the same geometry you learned in high school, the Pythagorean theorem, $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Use it for local analysis (e.g., measuring the distance between two buildings in the same city).

Example

- When you calculate distances on a sphere (unprojected), the math is computationally heavier and assumes a perfect ellipsoid.
- When you project to a local UTM zone, you are "flattening" that specific part of the earth, which allows for:
 - **+ Precision:** Minimal distortion within the specific zone.
 - **Consistency:** Easier integration with CAD / engineering data.
 - **Speed:** Linear math is faster for very large datasets (millions of points).

```
# Start with geographic points (WGS84)
pts_geo <- st_as_sf(data.frame(
  name = c("Point A", "Point B"),
  lon = c(-73.98, -73.95),
  lat = c(40.75, 40.78)
), coords = c("lon", "lat"), crs = 4326)

# Transform to a local projection (e.g., UTM
# Zone 18N for NYC)
pts_utm <- st_transform(pts_geo, crs = 32618)

# Calculate distance (will be in meters based
# on the UTM projection)
dist_planar <- st_distance(pts_utm[1,],
  pts_utm[2,])
```

Tips

- If you don't want to look up UTM zones manually, you can use the **crsuggest** package to find the best CRS for your specific coordinates:

```
# install.packages("crsuggest")  
library(crsuggest)  
  
suggested_crs <- suggest_crs(pts_geo)  
# This returns a tibble of the best  
# projections for your area!
```

Refs.:

<https://github.com/walkerke/crsuggest>

<https://www.pmassicotte.com/posts/2021-11-12-choosing-the-right-projection/index.html>



A medieval depiction of the Ecumene (1482, Johannes Schnitzer, engraver), constructed after the coordinates in Ptolemy's Geography and using his second map projection.

Image from: https://en.wikipedia.org/wiki/Map_projection

Some Conclusions

- To get the most accurate local measurements, you should transform your data from a global geographic system (like WGS84) to a Projected Coordinate System (PCS) like UTM (Universal Transverse Mercator).
- While geographic systems use degrees to measure a curved surface, projected systems use linear units (meters) on a flat plane, which is much more precise for distance calculations over smaller regions.

Next ...



- **Vector (sf)**
 - Represents the world as discrete objects with defined boundaries (points, lines, polygons).
 - Ideal for "Where is Texas A&M?"
- **Raster (terra)**
 - Represents the world as a continuous grid of cells (pixels).
 - Ideal for "What is the population density at this specific coordinate?"

- The importance of Raster-Vector Interactions lies in the ability to bridge two fundamentally different ways of representing the world.
- In data engineering and GIS, these interactions are the "glue" that allows us to perform complex spatial reasoning.
 - **Sampling (Contextual Enrichment):** It allows you to assign "environmental" or "demographic" data to a specific location. For example, taking a vector point (a store location, or its buffer) and extracting the raster values (population density from LandScan) to understand market reach.
 - **Zonal Statistics:** This is the process of calculating statistics (mean, maximum, sum) from a raster within the boundaries of a vector polygon. It's how we answer questions like, "What was the average rainfall for this specific county?"
 - **Precision Masking & Clipping:** Engineering workflows often involve massive datasets. Using vector boundaries (like a state or city limit) to "mask" a raster ensures that computational power is only spent on the area of interest, drastically improving processing speed.
 - **Hybrid Modeling:** Many physical processes are best modeled as rasters (e.g., flood plains, heat maps, storm surge), while human infrastructure is modeled as vectors (e.g., roads, levees, power lines). Interaction is required to see how infrastructure affects, or is affected by, environmental conditions.
 - **Temporal Analysis (Data Cubes):** By stacking rasters over time and "drilling" through them with a vector point, you can track how a specific location changes over years without needing to manually inspect every map.

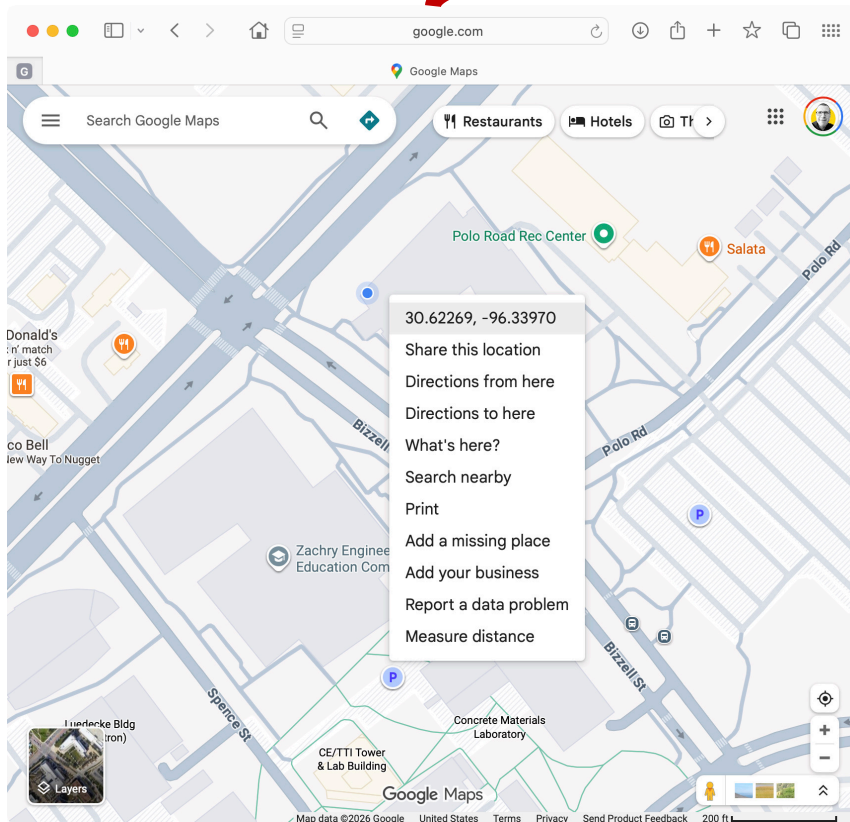
Assessing Risk

$$\text{Risk} = \underbrace{P(T)}_{\substack{\text{Hazard} \\ 0-1}} \times \underbrace{P(C|T)}_{\substack{\text{Vulnerability} \\ 0-1}} \times C$$

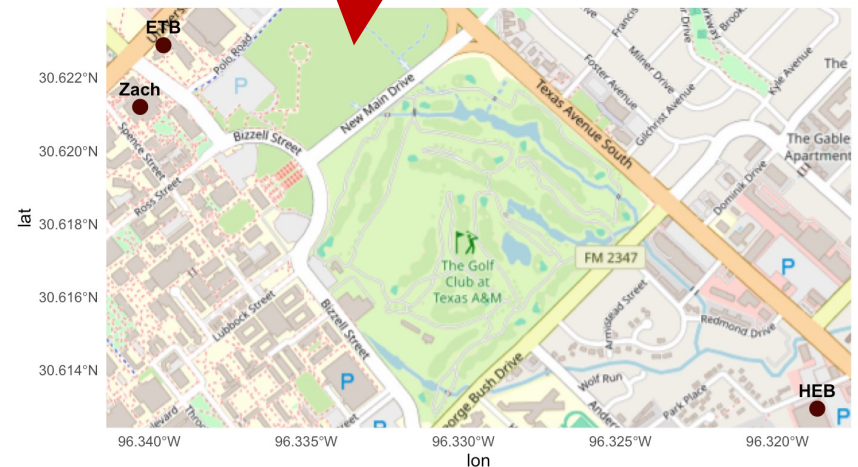
Exposure
0, 1, 2, 3 ...
people, houses, values, etc.

Example Points



```
locations <- data.frame(
  name = c("ETB", "Zach", "HEB"),
  lon = c(-96.3395, -96.3402, -96.3187),
  lat = c(30.6228, 30.6211, 30.6129))

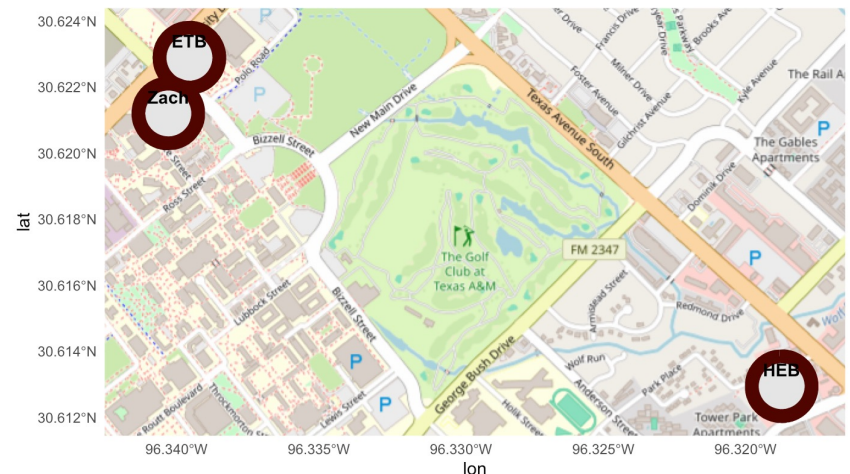
points_sf <-
  st_as_sf(locations,
    coords = c("lon", "lat"), crs = 4326)
```



Buffering

- A buffer expands a geometric feature (point, line, or polygon) into a new polygon that covers all areas within a specific distance (d) of the original feature.
- We use the sf package to wrap our locations in a "safety zone."
 - **Proximity Analysis:** "How many people live within 500 meters of this HEB?"
 - **Environmental Impact:** "Which dorms are within the noise radius of the construction site?"
 - **The Zombie Metric:** "If a zombie can smell a human from 50 feet, what is the 'Danger Zone' around the Zachry building?"
- Buffering works best when your CRS uses meters or feet rather than degrees.
- If you buffer by "1" in a degree-based system (WGS84), you'll create a bubble that spans an entire state!
- Example:

```
buffered_points <- st_buffer(points_sf, 100)
```



LandScan

- Program initiated at **Oak Ridge National Laboratory** (ORNL) in 1997 to address the need for improved estimates of population.
- Need for a more **realistic population distribution** that could capture where people are likely to be throughout the course of a day (not just residential locations as are reported in official **censuses**).
- The goal of capturing the diurnal variation of population is critical for a variety of analyses and actions including emergency preparedness.

- **LandScanUSA2021**

Weber, E., Moehl, J., Weston, S., Rose, A., Brelsford, C., & Hauser, T. (2022). LandScan USA 2021 [Data set]. Oak Ridge National Laboratory.

<https://landscan.ornl.gov>

- Download LandScan USA 2021 (*other years later*)
- Unzip it
 - This is the file we are looking for:
landscan-usa-2021-conus-night.tif

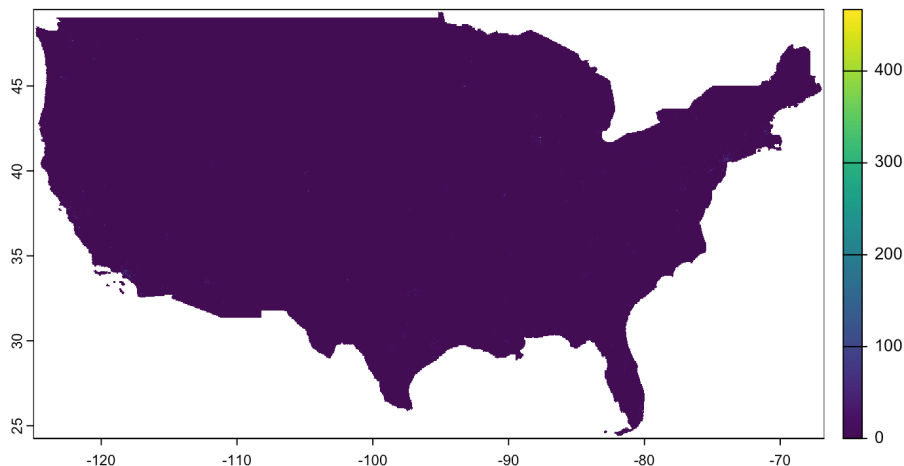
library(terra)

terra provides methods to manipulate geographic (spatial) data in "raster" and "vector" form.

library(sf)

landscan <- rast("/Users/alexander/Data/landscan-usa-2021-night/landscan-usa-2021-conus-night.tif")

plot(landscan)



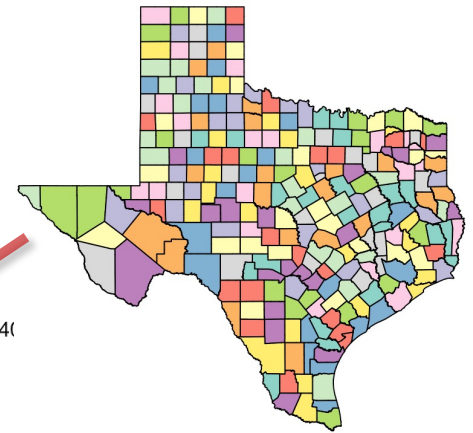
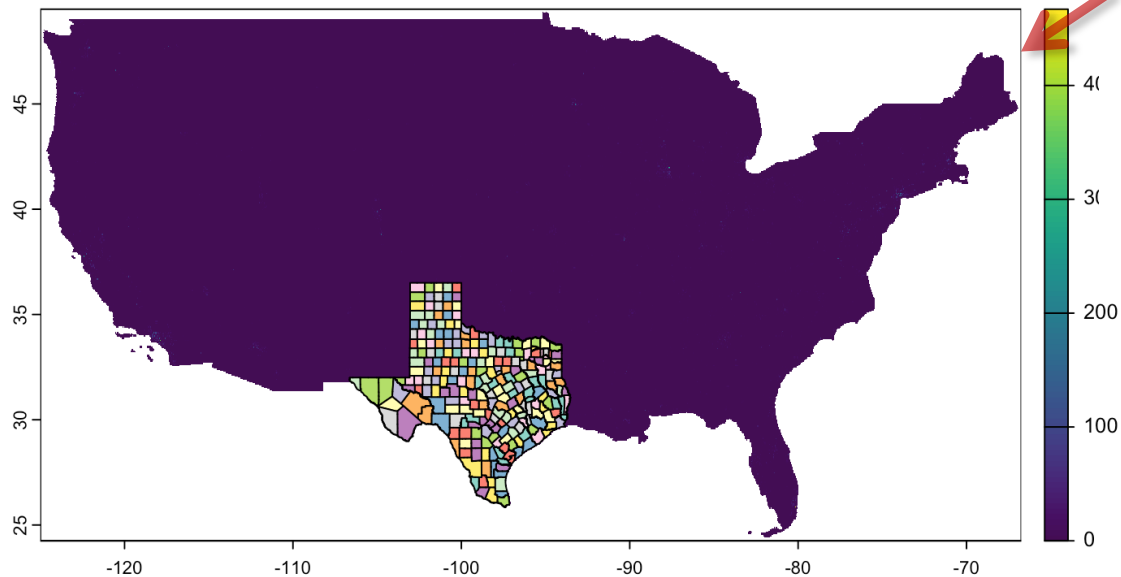
```
landscan <- rast("/Users/alexander/Data/landscan-usa-2021-night/landscan-usa-2021-conus-night.tif")
```

Dissolving

```
counties <- spatial_data %>% group_by(COUNTYFP) %>% summarise()
```

Reprojecting

```
counties <- counties %>% st_transform(st_crs(landscan))
```

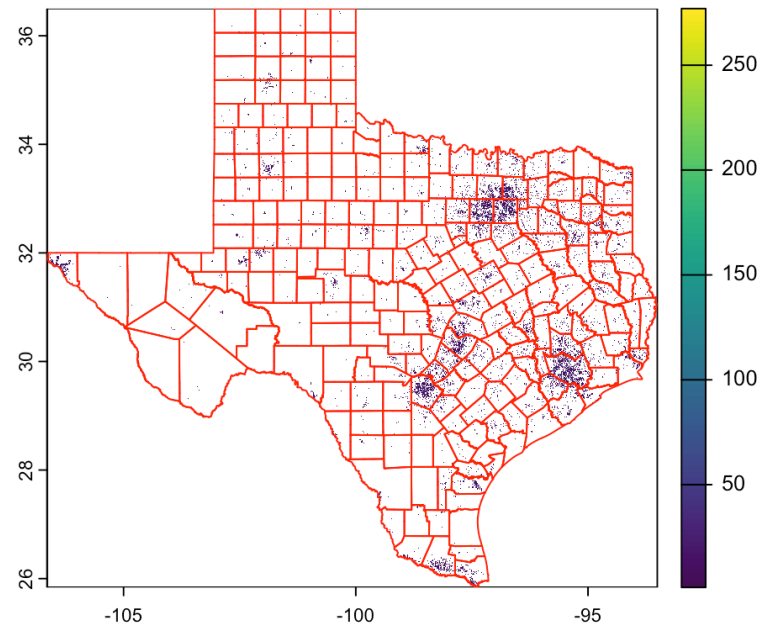
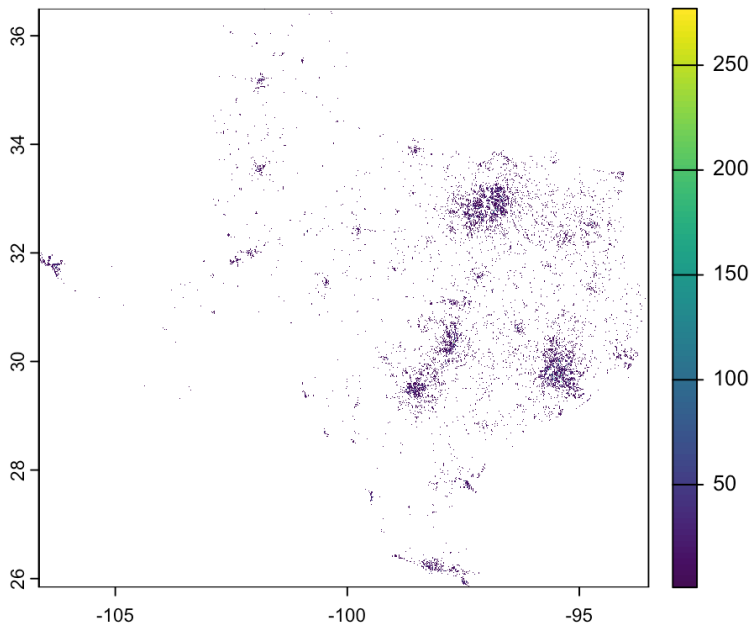


```
landscan <- rast("/Users/alexander/Data/landscan-usa-2021-night/landscan-usa-2021-conus-night.tif")
counties <- spatial_data %>% group_by(COUNTYFP) %>% summarise()
counties <- counties %>% st_transform(st_crs(landscan))
landscan <- crop(landscan, counties)
landscan <- mask(landscan, counties)
landscan[landscan[] == 0] = NA
```

→

Crop = bound box
Masking = really cropping

In this order the
file processes
much faster!



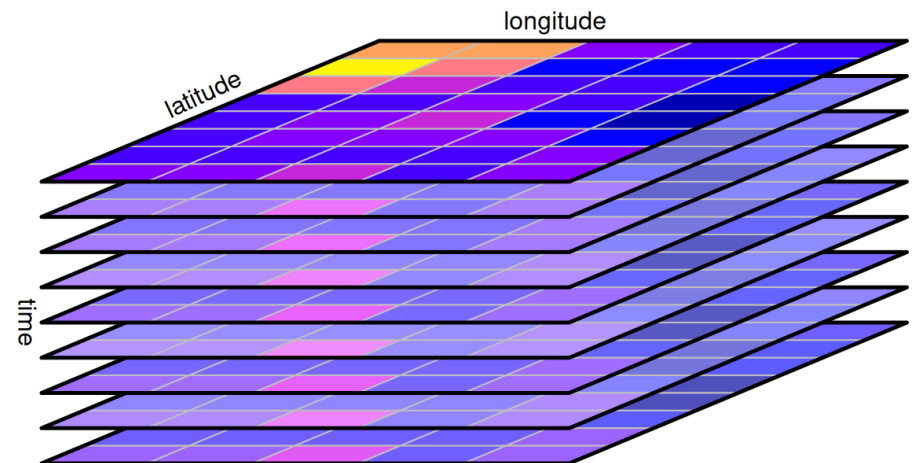
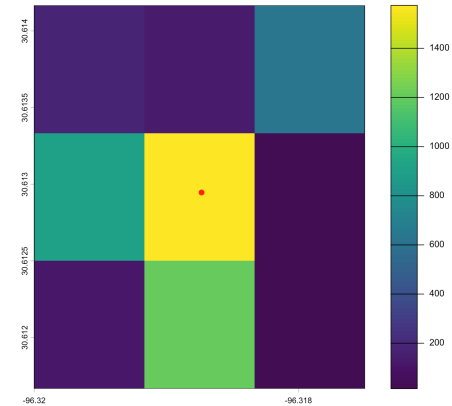
Data Cubes

(Slide 13, week 6)

```
cube <- c(ls2019day_mask, ls2020day_mask,  
          ls2021day_mask)  
  
names(cube) <- c("ls2019day", "ls2020day",  
                 "ls2021day")  
  
extract(cube, points_sf)
```

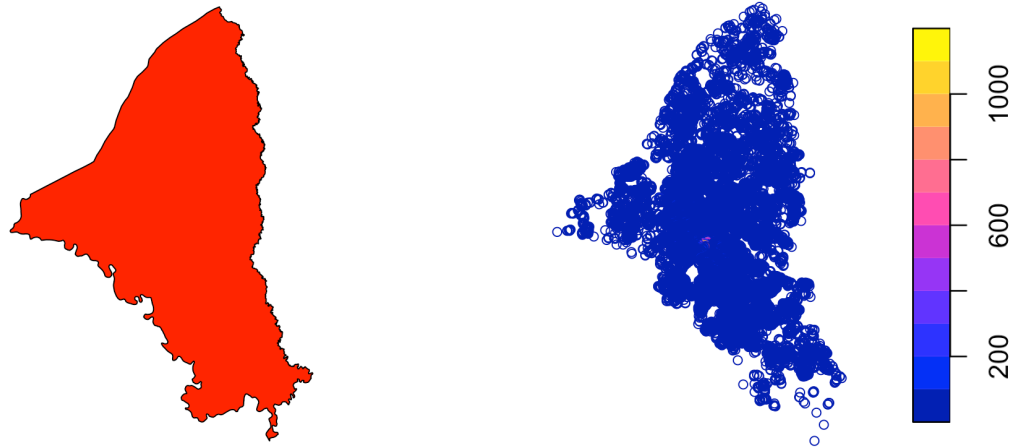
#	ID	ls2019day	ls2020day	ls2021day
# 1	1	55	55	346
# 2	2	0	0	507
# 3	3	1445	1445	1575

?



Also ...

```
landscan_pts <- as.points(landscan)  
landscan_pts <- st_as_sf(landscan_pts)  
colnames(landscan_pts)[1] <- "pop2021"  
landscan_pts_joined <- st_join(landscan_pts, counties)
```



In summary ...

```
my_vector <- spatial_data %>%  
  st_transform(st_crs(landscan))
```

} define your vector

```
landscan <- crop(landscan, my_vector)  
landscan <- mask(landscan, my_vector)  
landscan[landscan[] == 0] = NA
```

} prepare raster

```
landscan_pts <- as.points(landscan)  
landscan_pts <- st_as_sf(landscan_pts)  
colnames(landscan_pts)[1] <- "pop2021"
```

} convert raster to points (centroids)

```
landscan_pts_joined <- st_join(landscan_pts, my_vector) %>% na.omit()
```

} spatial join/spatial filter and clean points

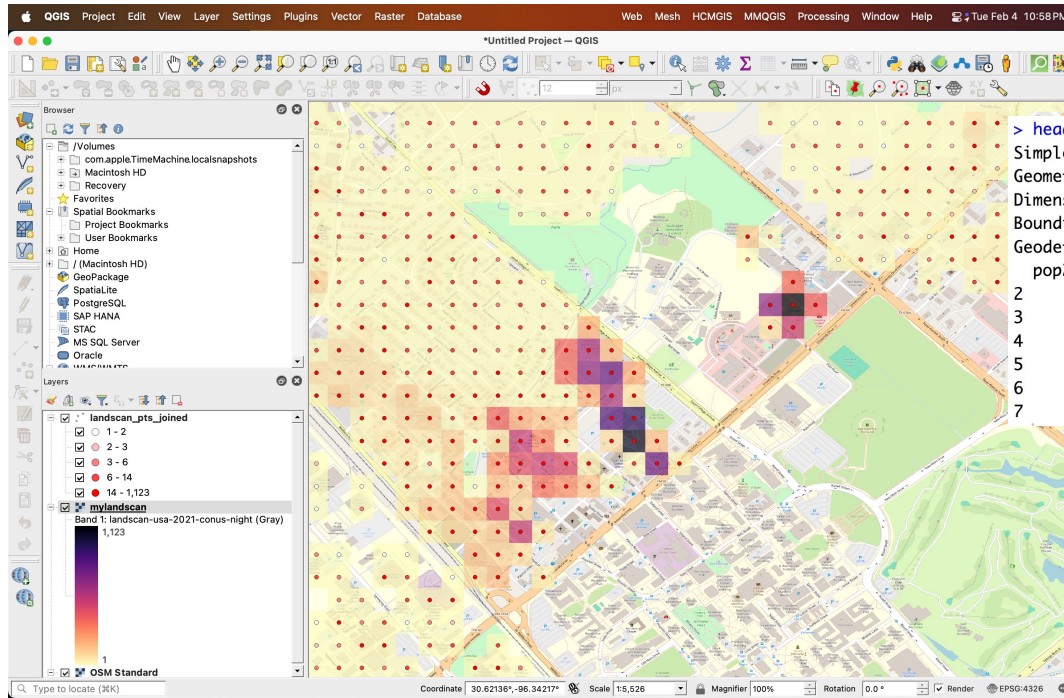
And ...

```
st_write(landscan_pts_joined, "landscan_pts_joined.shp")
```

} save shp with points

```
writeRaster(landscan, "mylandscan.tif", overwrite = TRUE)
```

} save your raster



```
> head(landscan_pts_joined)
```

Simple feature collection with 6 features and 4 fields

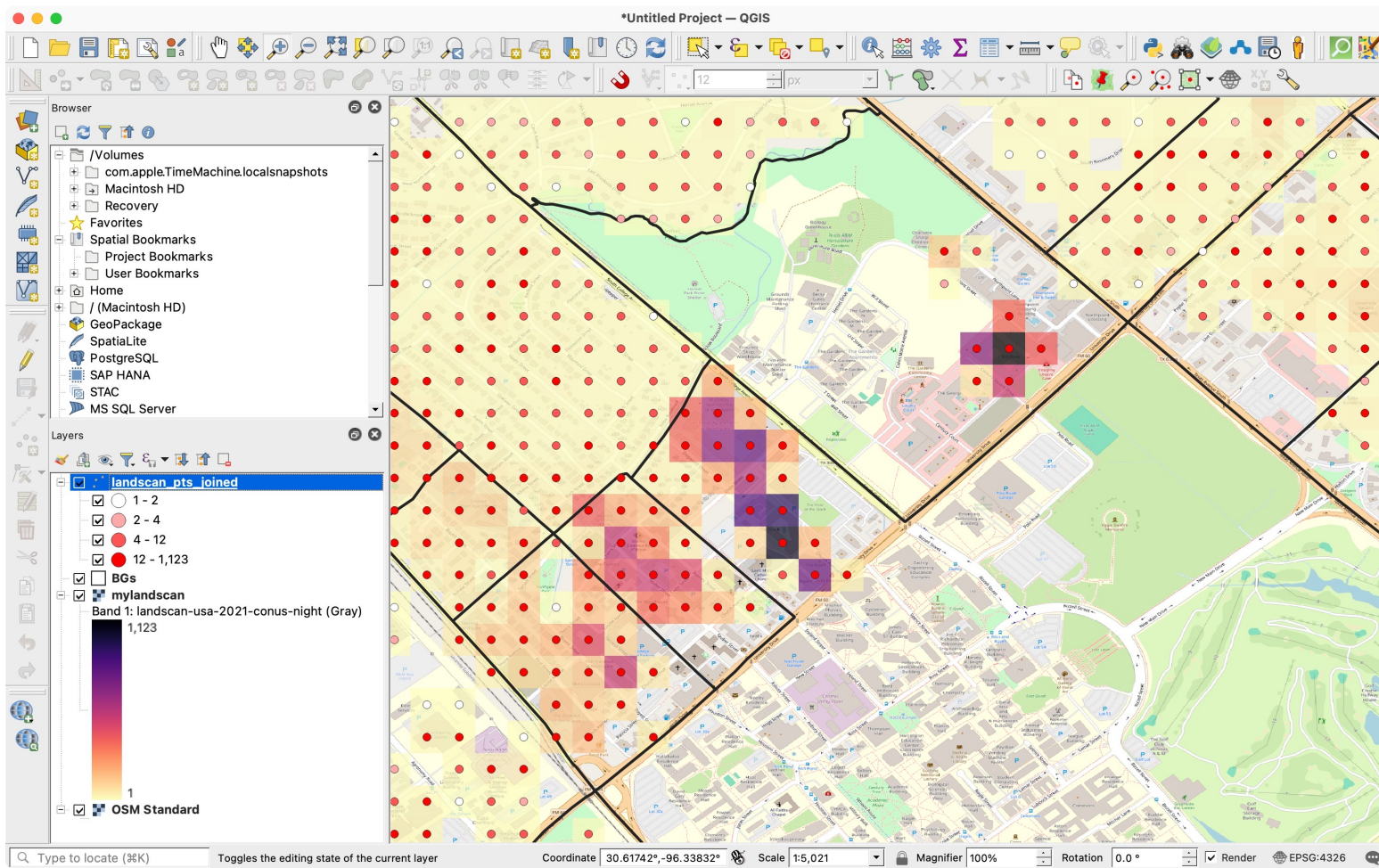
Geometry type: POINT

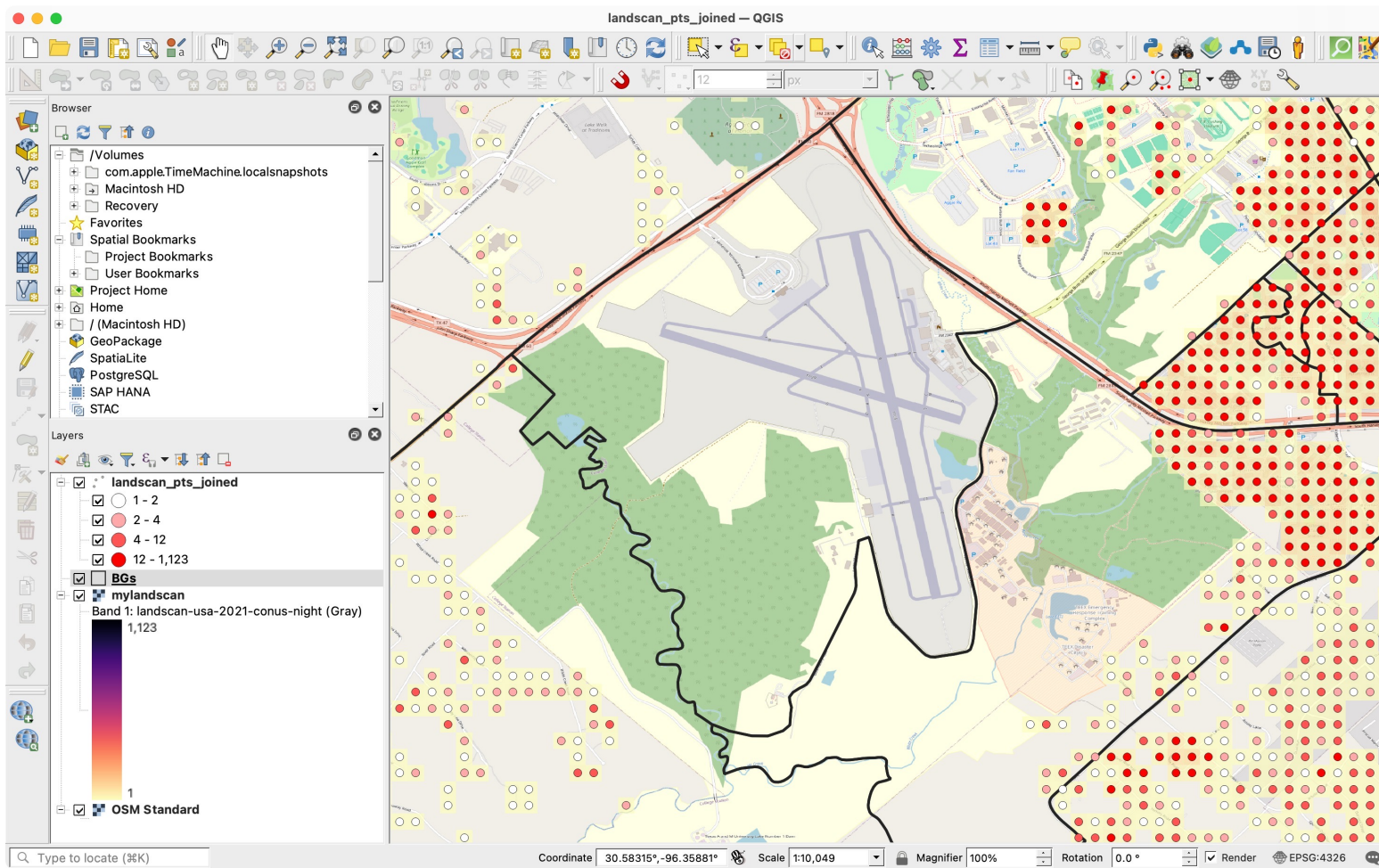
Dimension: XY

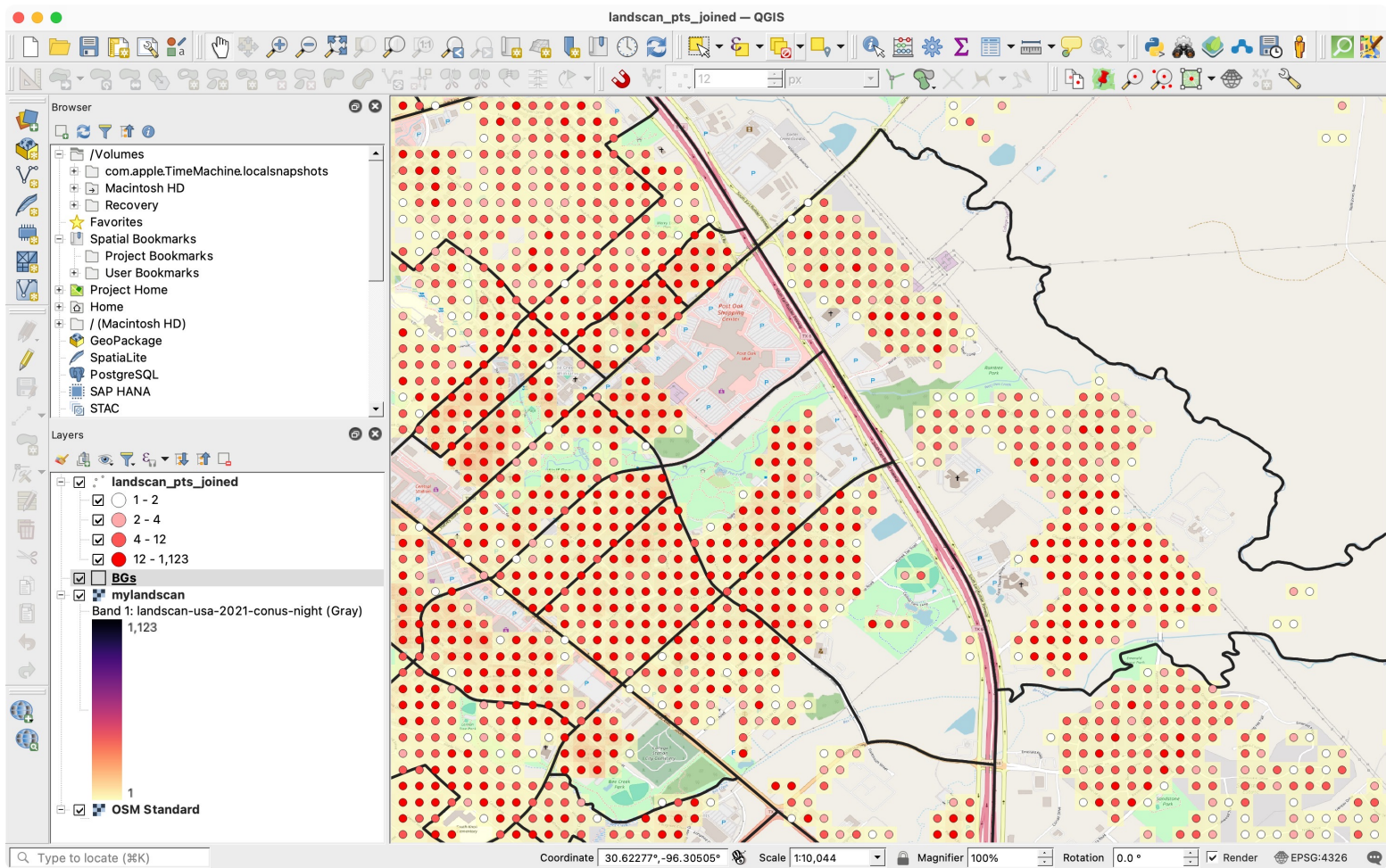
Bounding box: xmin: -96.26458 ymin: 30.96125 xmax: -96.24625 ymax: 30.96792

Geodetic CRS: WGS 84

	pop2021	COUNTYFP	GEOID	POPe	geometry
2	1	48041	480410001031	2230	POINT (-96.25125 30.96792)
3	1	48041	480410001031	2230	POINT (-96.24625 30.96542)
4	2	48041	480410001031	2230	POINT (-96.26042 30.96375)
5	2	48041	480410001031	2230	POINT (-96.26458 30.96208)
6	1	48041	480410001031	2230	POINT (-96.26375 30.96208)
7	1	48041	480410001031	2230	POINT (-96.24875 30.96125)







Homework Assignment

1. Create a data frame with some points.
2. Estimate distances (and verify it, can use Google Maps for it).
3. Estimate the population at the location, and descriptive statistics the population around
 - Create a population data cube for at least 5 years period.
 - Your choice of buffer, briefly justify, come up with some idea.
 - Briefly explain/compare how population change over time (advantages and disadvantages of using the buffer).
4. Include a data analysis workflow and explain it with a graph, diagram, or pseudo-code.
 - From mess to meaning ...
5. Submission on Canvas
 - Due Thu 26 before class (aka 12pm)

